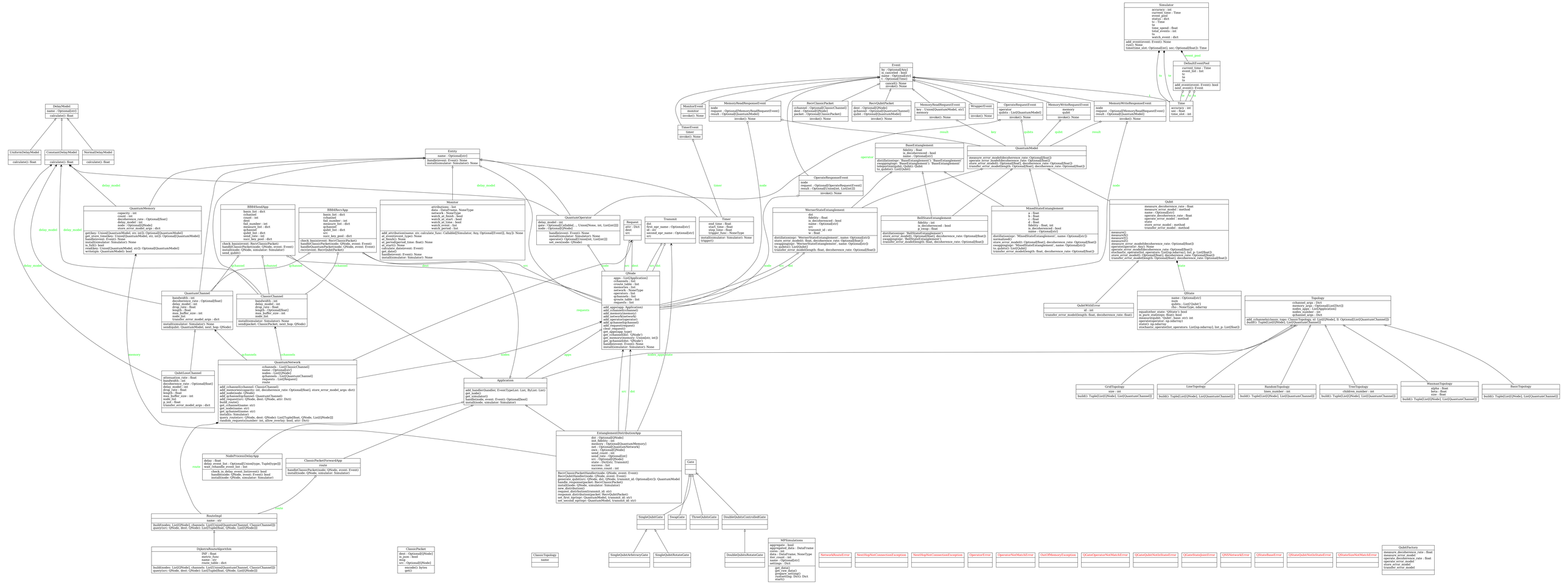
Simulator
accuracy : int
current_time : Time
events : Heap
stopped : dict
tc : Time
ts : Time
time_speed : float
total_events : int
ts
watch_event : dict
add_event(event: Event): None
run(): None
timeline_slot: Optional[int], sec: Optional[float]): Time
event_pool
DefaultEventPool
current_time : Time
event_list : list
ts
tc
add_event(event: Event): bool
next_event() : Event
Time
accuracy : int
sec : float
time_slot : int
Event
by : Optional[Any]
is_canceled : bool
priority : Optional[int]
t : Optional[Time]
cancel(): None
invoke(): None
MonitorEvent
monitor
invoke(): None
TimerEvent
timer
invoke(): None
MemoryReadResponseEvent
node
request : Optional[MemoryReadRequestEvent]
result : Optional[QuantumModel]
invoke(): None
RecvClassicPacket
cchannel : Optional[ClassicChannel]
dest : Optional[QNode]
packet : Optional[ClassicPacket]
invoke(): None
RecvQubitPacket
dest : Optional[QNode]
qchannel : Optional[QuantumChannel]
qubit : Optional[QuantumModel]
invoke(): None
MemoryReadRequestEvent
key : Union[QuantumModel, str]
memory
invoke(): None
WrapperEvent
invoke(): None
OperateRequestEvent
operator
qubits : List[QuantumModel]
invoke(): None
MemoryWriteRequestEvent
memory
qubit
invoke(): None
MemoryWriteResponseEvent
node
request : Optional[MemoryReadRequestEvent]
result : Optional[QuantumModel]
invoke(): None
result
key
qubits
qubit
result
node
DelayModel
name : Optional[str]
calculate() : float
UniformDelayModel
calculate() : float
ConstantDelayModel
calculate() : float
NormalDelayModel
calculate() : float
Entity
name : Optional[str]
handle(event: Event): None
install(simulator: Simulator): None
delay_model
delay_model
delay_model
delay_model
delay_model
delay_model
QuantumMemory
capacity : int
count : int
decoherence_rate : Optional[float]
delay_model : int
node : Optional[QNode]
store_error_model_args : dict
get(key: Union[QuantumModel, str], ts=0): Optional[QuantumModel]
get_store_time(key: Union[QuantumModel, str], ts=0): Optional[QuantumModel]
handle(event: Event): None
install(simulator: Simulator): None
put(qubit): None
read(key: Union[QuantumModel, str]): Optional[QuantumModel]
write(qm: QuantumModel): bool
BBM84SendApp
basis_list : dict
cchannel
count : int
cq_dict
final_number : int
measure_list : dict
qchannel
qubit_list : dict
sifting_cb
succ_key_pool : dict
check_basis(event: RecvClassicPacket)
handle(ClassicPacket|node: QNode, event: Event)
install(node: QNode, simulator: Simulator)
send_qubit()
BBM84RecvApp
basis_list : dict
cchannel
count : int
measure_list : dict
qchannel
qubit_list : dict
sifting_cb
succ_key_pool : dict
check_basis(event: RecvClassicPacket)
handle(ClassicPacket|node: QNode, event: Event)
handle(QuantumPacket|node: QNode, event: Event)
recv(event: RecvQubitPacket)
Monitor
attributes : list
data : DataFrame, NoneType
network : NoneType
watch_at_finish : bool
watch_at_start : bool
watch_at_time : bool
watch_event : list
watch_period : list
add_attribute(name: str, calculate_func: Callable[Simulator, Any, Optional[Event]], Any]): None
at_event(event_type): None
at_finish(): None
at_period(period, time: float): None
at_start(): None
calculate_data(event: Event)
get_data()
handle(event: Event): None
install(simulator: Simulator): None
QuantumOperator
delay_model : int
gate : Optional[Callable[... , Union[None, int, List[int]]]]
node : Optional[QNode]
handle(event: Event): None
install(simulator: Simulator): None
operate(): Optional[Union[int, List[int]]]
set_ownlode: QNode)
Request
attr : Dict
dest
src
Transmit
dst
first_epr_name : Optional[str]
id : str
second_epr_name : Optional[str]
src
install(simulator: Simulator): None
Timer
end_time : float
start_time : float
step_time : float
trigger_func : NoneType
install(simulator: Simulator): None
(trigger)
OperateResponseEvent
node
request : Optional[OperateRequestEvent]
result : Optional[Union[int, List[int]]]
invoke(): None
BaseEntanglement
fidelity : float
is_decoherenced : bool
name : Optional[str]
distillation(np: 'BaseEntanglement'): 'BaseEntanglement'
swapping(np: 'BaseEntanglement'): 'BaseEntanglement'
teleport(qubit: Qubit): Qubit
to_qubits() : List[Qubit]
QuantumModel
measure_error_model(decoherence_rate: Optional[float])
operate_error_model(decoherence_rate: Optional[float])
store_error_model(): Optional[float], decoherence_rate: Optional[float])
transfer_error_model(length: Optional[float], decoherence_rate: Optional[float])
operate
QuantumChannel
bandwidth : int
decoherence_rate : Optional[float]
delay_model : int
drop_rate : float
length : float
max_buffer_size : int
node list
transfer_error_model_args : dict
install(simulator: Simulator): None
send(qubit: QuantumModel, next_hop: QNode)
ClassicChannel
bandwidth : int
delay_model : int
drop_rate : float
length : Optional[float]
max_buffer_size : int
node list
install(simulator: Simulator): None
send(packet: ClassicPacket, next_hop: QNode)
cchannel
cchannel
cchannel
cchannel
cchannel
cchannel
dest
src
node
src
dest
dst
src
WernerStateEntanglement
dst
fidelity : float
is_decoherenced : bool
name : Optional[str]
src
transmit_id : str
w : float
distillation(np: 'WernerStateEntanglement', name: Optional[str])
store_error_model(): float, decoherence_rate: Optional[float])
swapping(np: 'WernerStateEntanglement', name: Optional[str])
transfer_error_model(length: float, decoherence_rate: Optional[float])
BellStateEntanglement
fidelity : int
is_decoherenced : bool
p_swap : float
distillation(np: 'BellStateEntanglement')
store_error_model(): float, decoherence_rate: Optional[float])
swapping(np: 'BellStateEntanglement')
transfer_error_model(length: float, decoherence_rate: Optional[float])
MixedStateEntanglement
a : float
b : float
c : float
d : float
fidelity : float, int
is_decoherenced : bool
name : Optional[str]
distillation(np: 'MixedStateEntanglement', name: Optional[str])
normalized()
store_error_model(): Optional[float], decoherence_rate: Optional[float])
swapping(np: 'MixedStateEntanglement', name: Optional[str])
to_qubits() : List[Qubit]
transfer_error_model(length: float, decoherence_rate: Optional[float])
Qubit
measure_decoherence_rate : float
measure_error_model : method
name : Optional[str]
operate_decoherence_rate : float
operate_error_model : method
state
store_error_model : method
transfer_error_model : method
measure()
measure(b)
measure(d)
measure(r)
measure_error_model(decoherence_rate: Optional[float])
operate(operator: 'Operator')
operate_error_model(decoherence_rate: Optional[float])
stochastic_operate(list_operators: List[np.ndarray], list_p: List[float])
store_error_model(): Optional[float], decoherence_rate: Optional[float])
transfer_error_model(length: Optional[float], decoherence_rate: Optional[float])
from
QNode
apps : List[Application]
cchannels : list
crosslc_table : list
name : str
network : NoneType
operators : list
qchannels : list
qmemory : QuantumMemory
requests : list
add_app(app: Application)
add_cchannel(cchannel)
add_qchannel(qchannel)
add_set(network)
add_operator(operator)
add_qmemory(memory)
add_request(request)
clear_request()
get_cchannel(str)
get_cchannel(dst: QNode)
get_memory(memory: Union[str, int])
get_qchannel(dst: QNode)
handle(event: Event): None
install(simulator: Simulator): None
requests
nodes
apps
nodes_applicable
QubitWithError
id : int
transfer_error_model(length: float, decoherence_rate: float)
QState
name : Optional[str]
num
qubits : List[Qubit]
rho : NoneType, ndarray
equal(other_state: 'QState') :bool
is_pure_state() : bool
operate(qubit: Qubit, base: str) :int
operate(rho: ndarray, op:ndarray)
state(): np.ndarray
stochastic_operate(list_operators: List[np.ndarray], list_p: List[float])
Topology
cchannel_args : Dict
children_name : Optional[List[Int]]
nodes_apps : List[Application]
nodes_number : int
qchannel_args : Dict
add_cchannel(cclassic: topo: ClassicTopology, n1: List[QNode], l: Optional[List[ClassicChannel]])
build(): Tuple[List[QNode], List[QuantumChannel]]
QubitLossChannel
attenuation_rate : float
bandwidth : int
decoherence_rate : Optional[float]
delay_model : int
drop_rate : float
length : float
max_buffer_size : int
node list
p_loss : float
transfer_error_model_args : dict
QuantumNetwork
cchannels : List[ClassicChannel]
name : Optional[str]
nodes : List[QNode]
qchannels : List[QuantumChannel]
requests : List[Request]
route
add_cchannel(cchannel: ClassicChannel)
add_qchannel(node: int, decoherence_rate: Optional[float], store_error_model_args: dict)
add_node(node: QNode)
add_qchannel(qchannel: QuantumChannel)
add_request(src: QNode, dest: QNode, attr: Dict)
build_route()
get_cchannel(name: str)
get_node(name: str)
get_qchannel(name: str)
install(n: Simulator)
query_route(src: QNode, dest: QNode): List[Tuple[float, QNode, List[QNode]]]
random_requests(number: int, allow_overlay: bool, attr: Dict)
channels
channels
Application
add_handler(handler, EventTypeList: List, By[int: List)
get_node()
get_simulator()
install(event: Optional[Event]): Optional[Bool]
install(node, simulator: Simulator)
src
dst
EntanglementDistributionApp
dst : Optional[QNode]
name : str
memory : Optional[QuantumMemory]
net : Optional[QuantumNetwork]
node : Optional[QNode]
send_count : int
send_rate : Optional[int]
simple_swap : bool
state : Dict[str, Transmit]
success : list
success_count : int
RecvClassicPacketHandler(node: QNode, event: Event)
RecvQubitPacket(node: QNode, event: Event)
generate_epl(str): QNode, dst: QNode, transmit_id: Optional[str]): QuantumModel
handle_responser(packet: RecvClassicPacket)
install(node: QNode, simulator: Simulator)
new_distillation()
request_distribution(transmit_id: str)
response_distribution(packet: RecvQubitPacket)
set_first_epr(epr: QuantumModel, transmit_id: str)
set_second_epr(epr: QuantumModel, transmit_id: str)
NodeProcessDelayApp
delay : float
delay_event_list : Optional[Union[type, Tuple[type]]]
send_rebuild_event_list : int
check_is_delay_event_list(event): bool
handle(event: Event): Optional[Bool]
install(node: QNode, simulator: Simulator)
ClassicPacketForwardApp
route
handle(ClassicPacket|node: QNode, event: Event)
install(node: QNode, simulator: Simulator)
route
route
RouteImpl
name : str
build(nodes: List[QNode], channels: List[Union[QuantumChannel, ClassicChannel]])
query(src: QNode, dest: QNode): List[Tuple[float, QNode, List[QNode]]]
DijkstraRouteAlgorithm
INF : float
metric_func
name : str
route_table : dict
build(nodes: List[QNode], channels: List[Union[QuantumChannel, ClassicChannel]])
query(src: QNode, dest: QNode): List[Tuple[float, QNode, List[QNode]]]
ClassicPacket
dest : Optional[QNode]
is_loss : bool
msg : dict
src : Optional[QNode]
encode(): bytes
get()
Gate
SingleQubitGate
SwapGate
ThreeQubitGate
DoubleQubitsControlledGate
ClassicTopology
name
SingleQubitArbitrageGate
SingleQubitRotateGate
DoubleQubitsRotateGate
MPSimulations
aggregate : bool
aggregated_data : DataFrame
data
data : DataFrame, NoneType
fiber_count : int
name : Optional[str]
settings : Dict
get_data()
get_raw_data()
prepare_setup()
run(Dict | Dict): Dict
start()
NetworkRouteError
NextHopNotConnectionException
NextHopNotConnectionException
OperatorError
OperatorNotMatchError
OutOfMemoryException
QGateOperatorNotMatchError
QGateQubitNotInStateError
QGateStateJointError
QNNetworkError
QStateBaseError
QStateQubitNotInStateError
QStateSizeNotMatchError
QubitFactory
measure_decoherence_rate : float
measure_error_model
operate_decoherence_rate : float
operate_error_model
store_error_model
transfer_error_model
GridTopology
size : int
build(): Tuple[List[QNode], List[QuantumChannel]]
LineTopology
build(): Tuple[List[QNode], List[QuantumChannel]]
RandomTopology
lines_number : int
build(): Tuple[List[QNode], List[QuantumChannel]]
TreeTopology
children_number : int
build(): Tuple[List[QNode], List[QuantumChannel]]
WaxmanTopology
alpha : float
beta : float
size : float
build(): Tuple[List[QNode], List[QuantumChannel]]
BasicTopology
build(): Tuple[List[QNode], List[QuantumChannel]]